

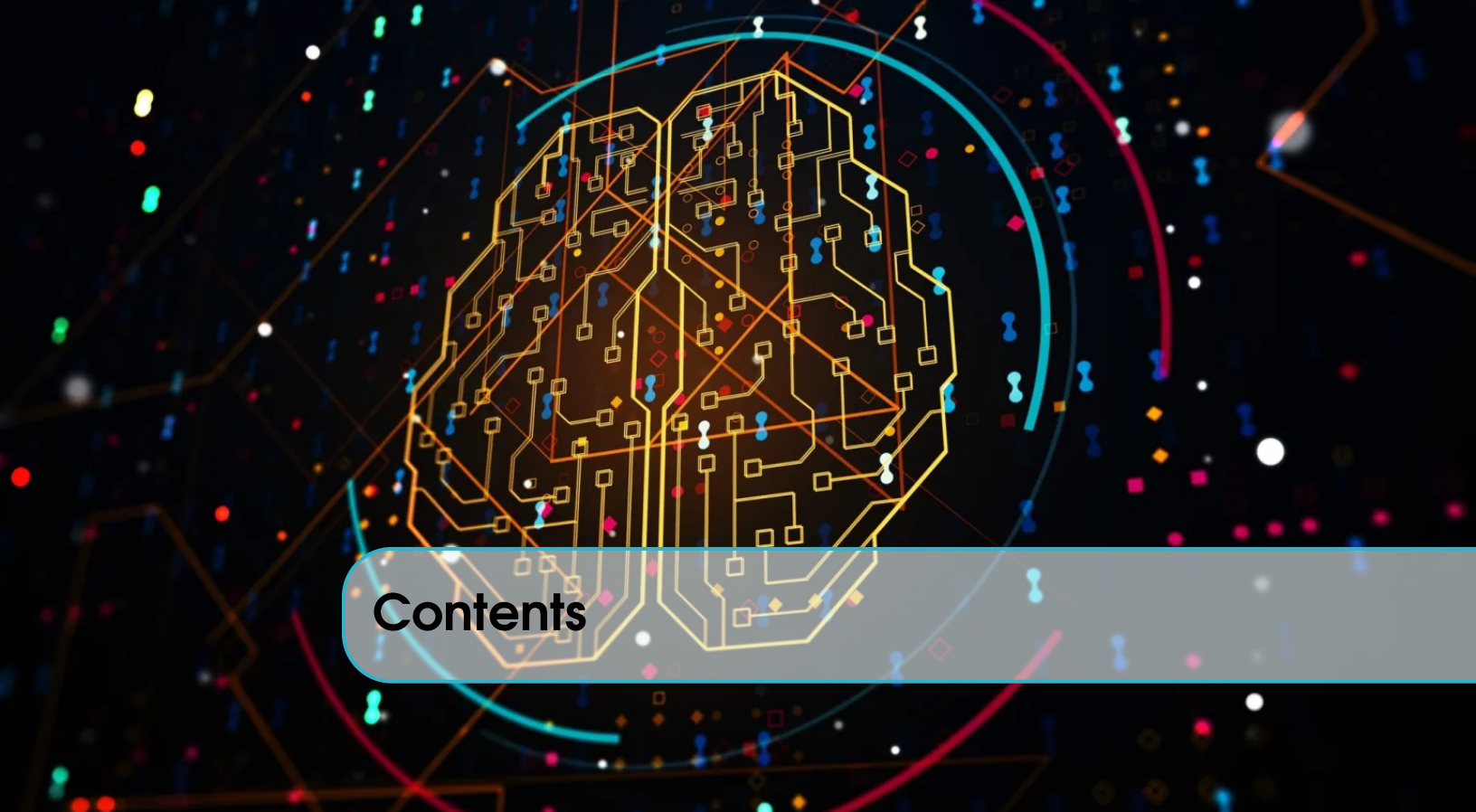
The background of the entire cover is a vibrant, multi-colored cosmic nebula. It features swirling clouds of gas and dust in shades of deep blue, magenta, orange, and yellow. Several bright stars are visible, some with prominent diffraction spikes. The nebula's structure is complex, with dense, clumpy regions and more diffuse, wispy areas.

Advanced Machine Learning

La Voglia di Morire

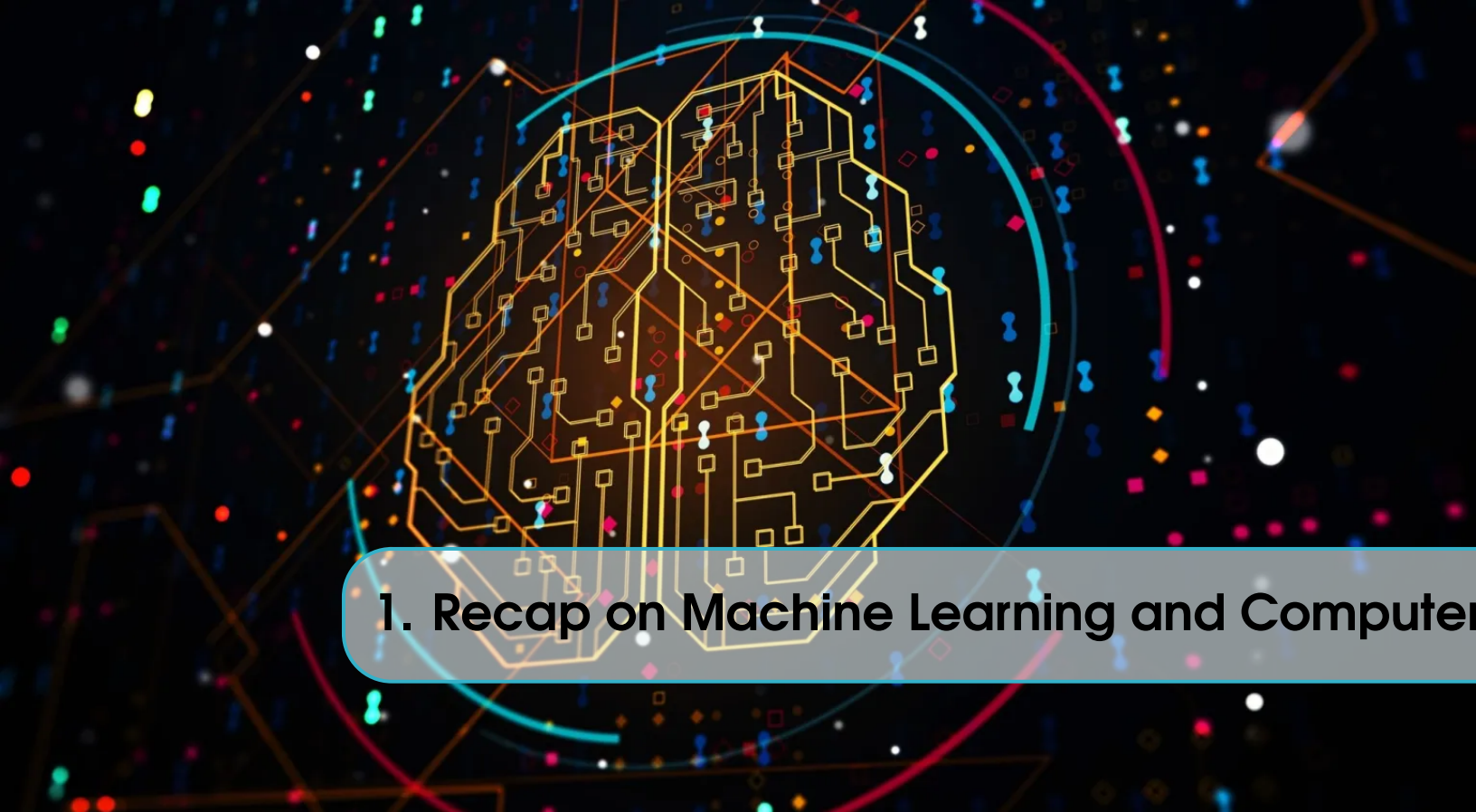
Vincenzo Salvatore

Raffaele Mercurio



Contents

1	Recap on Machine Learning and Computer Vision	5
1.1	Introduction to the Core Concepts	5
1.2	Image Classification	6
1.3	Parametric Models and Gradient-Based Optimization	6
1.3.1	Linear Regression ($f : \mathbb{R}^D \rightarrow \mathbb{R}$)	7
1.3.2	Logistic Regression ($f : \mathbb{R}^D \rightarrow [0, 1]$)	7
1.3.3	Softmax Classifier ($f : \mathbb{R}^D \rightarrow \{0, \dots, K\}$)	7
1.4	Advanced Optimization and Regularization	9
1.4.1	Regularization	9
1.4.2	Stochastic Gradient Descent (SGD)	10
1.5	Backpropagation and Neural Networks	10
1.5.1	Backpropagation	10
1.5.2	Deeper Neural Networks	11
1.5.3	The Role of Activation Functions	11



1. Recap on Machine Learning and Computer

1.1 Introduction to the Core Concepts

The contents of this chapter are largely based on the introductory slides of the course [1].

Key Word

- According to Arthur Samuel (1959), **Machine Learning** is the field of study that gives computers the ability to learn without being explicitly programmed.
- Tom Mitchell (1998) defined the well-posed learning problem: a computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

Machine learning is heavily applied in **Computer Vision**, which can be viewed from three main perspectives: as a science (exploring the computational model of human perception), as engineering (building systems that perceive the world), and through its applications.

Example

Some typical **Computer Vision Applications** include:

- **Medical imaging:** Supporting medical diagnosis and visualization.
- **Surveillance:** Tracking people in airports or train stations.
- **Autonomous driving:** Lane-keeping and pre-crash intervention.

1.2 Image Classification

A core task in computer vision is **Image Classification**: assuming a given set of discrete labels (e.g., {dog, cat, truck, plane}), the goal is to map an input digital image to one of these labels.

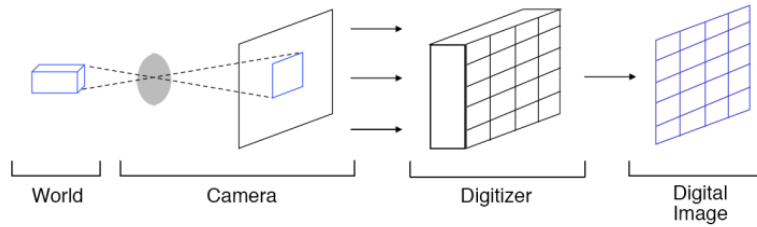


Figure 1.1: Illustration of the (pinhole) camera imaging process.

Note

The Imaging Process: An image is typically formed via a (pinhole) camera model and then passed through a digitizer to obtain a digital image, which the computer interprets as a large multi-dimensional array of numbers.

1.3 Parametric Models and Gradient-Based Optimization

The majority of machine learning models can be seen as **Parametric Models**. A parametric model takes an input x and uses a set of parameters (or weights) W and biases b to produce an output prediction z :

$$z = f(x, W) = Wx + b$$

While the exact nature of the prediction depends on the task, what unifies these models is the **Optimization Process**. We define a Loss function L that measures the error of our predictions, and we iteratively adjust the weights using **Gradient Descent**:

$$W^{t+1} = W^t - \alpha \nabla_W L$$

where α is the learning rate.

By simply changing the *activation function* applied to the linear score z and the corresponding *loss function*, we obtain different classical models: Linear Regression, Logistic Regression, and the Softmax Classifier.

1.3.1 Linear Regression ($f : \mathbb{R}^D \rightarrow \mathbb{R}$)

Linear Regression is used when the target variable is continuous. We use the identity activation function (i.e., no activation), so the predicted output $\hat{y}$ is exactly the linear score:

$$\hat{y} = z = Wx + b$$

We optimize this model by minimizing the **Mean Squared Error (MSE)** loss:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

1.3.2 Logistic Regression ($f : \mathbb{R}^D \rightarrow [0, 1]$)

Logistic Regression is used for binary classification. We map the linear score z to a probability between 0 and 1 using the **Sigmoid** activation function $\sigma(z)$:

$$P(Y = 1|X = x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The appropriate loss function for this model is the **Binary Cross-Entropy Loss (BCE)**. For a dataset of N samples, it is defined as:

$$L = -\frac{1}{N} \sum_{i=1}^N \left[\underbrace{y_i \log(\sigma(z_i))}_{\text{Active if } y=1} + \underbrace{(1 - y_i) \log(1 - \sigma(z_i))}_{\text{Active if } y=0} \right]$$

Note

Understanding the BCE Formula: This equation uses a clever mathematical trick to combine two mutually exclusive cases into a single line:

- When the true class is $y_i = 1$, the second term $(1 - 1) = 0$ disappears, and the loss evaluates only $-\log(\sigma(z_i))$, penalizing the model if it predicts a low probability.
- When the true class is $y_i = 0$, the first term 0 disappears, and the loss evaluates only $-\log(1 - \sigma(z_i))$, penalizing the model if it predicts a high probability for class 1.

1.3.3 Softmax Classifier ($f : \mathbb{R}^D \rightarrow \{0, \dots, K\}$)

The Softmax Classifier generalizes Logistic Regression to multiple, mutually exclusive classes. It maps the raw scores z into a valid probability distribution over the classes using the **Softmax function**:

$$P(Y = k|X = x_i) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Note

Why Softmax? We want to interpret raw classifier scores as *probabilities*. The Softmax function transforms scores such that they are strictly positive and sum to 1.

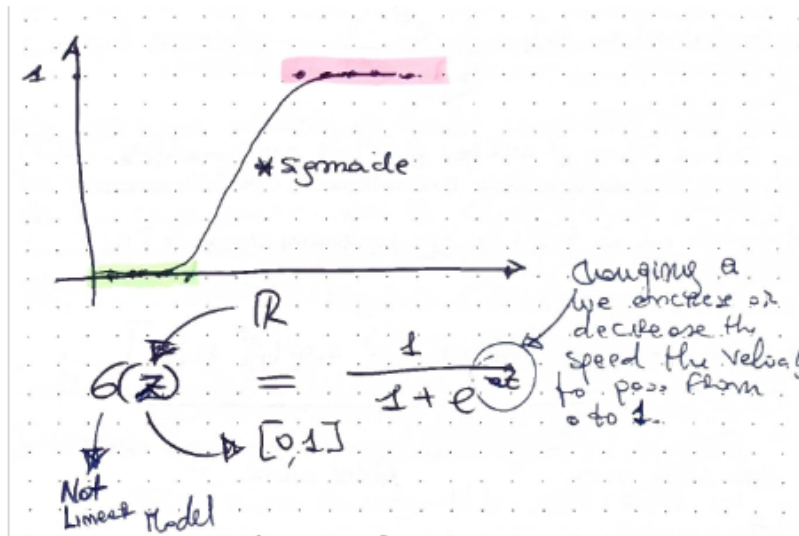


Figure 1.2: Illustration of the Sigmoid function in Logistic Regression.

Example

For an input image of size $32 \times 32 \times 3$ (e.g., CIFAR-10), the flattened array x has 3072 numbers. If we are classifying into 10 categories, the weight matrix W will have dimensions 10×3072 , producing a 10×1 vector of raw class scores before the Softmax is applied.

We then optimize the model using the **Categorical Cross-Entropy Loss**, often referred to as the Negative Log-Likelihood (NLL). Over the entire dataset of N samples, expanding the Softmax probability formula, the loss becomes:

$$NLL(W, X) = - \sum_{i=1}^N \left[\underbrace{W_{y_i}^T x_i}_{\text{Punteggio della classe giusta}} - \underbrace{\log \left(\sum_{k=1}^K \exp(W_k^T x_i) \right)}_{\text{Somma dei punteggi delle altre classi}} \right]$$

Note

Understanding the Softmax NLL Formula: By looking at the expanded formula, we see that the optimizer tries to maximize the linear score $W_{y_i}^T x_i$ of the *correct class* y_i , while simultaneously pushing down the log-sum of the scores of *all available classes*. This creates a competitive environment where increasing the score of the correct class implicitly decreases the relative probability of the others.

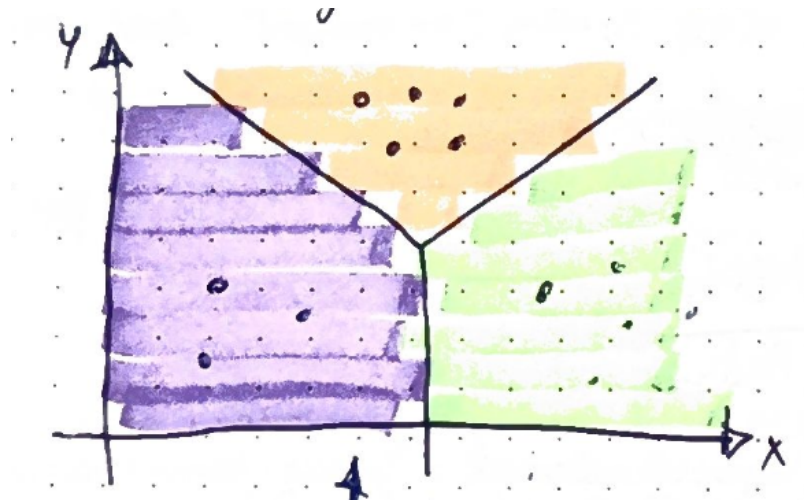


Figure 1.3: Illustration of the Sigmoid function in Logistic Regression.

1.4 Advanced Optimization and Regularization

1.4.1 Regularization

The primary goal of Regularization is to prevent the model from becoming too complex and overfitting the training data (i.e., we use it to combat **High Variance**). We achieve this by adding a penalty term $R(W)$, based only on the model's parameters, directly to the Data Loss.

The new Total Loss becomes:

$$L_{total}(W, X) = \underbrace{L(W, X)}_{\text{Data Loss}} + \underbrace{\lambda R(W)}_{\text{Regularization Term}}$$

Note

When and how is it applied?

Regularization is applied *during training*. By redefining the Total Loss, we also change its gradient. When we apply Gradient Descent, the algorithm must now minimize both the data error AND the complexity of the weights. For example, the gradient of an L2 penalty λW^2 is $2\lambda W$, meaning Gradient Descent will actively shrink the weights at every step (Weight Decay).

The hyperparameter λ acts as the “manipulator” of complexity:

- $\lambda = 0$: We ignore complexity. The model relies entirely on Data Loss (Risk of **Overfitting**).
- $\lambda \rightarrow \infty$: The model becomes “lazy”. The penalty is so high that Gradient Descent brings all weights to zero (Risk of **Underfitting**).

The two most classic mathematical formulations are:

- **L2 Regularization (Ridge)**: $R(W) = \sum W^2$. It punishes larger weights more severely, forcing the network to distribute the “weight” across all features rather than relying heavily on just a few. It shrinks weights but rarely brings them exactly to zero.
- **L1 Regularization (Lasso)**: $R(W) = \sum |W|$. It punishes weights constantly. This has the mathematical effect

of bringing less important weights exactly to zero, acting as an automatic *Feature Selection* mechanism.

Note: In deep neural networks, we also use architectural regularizers like Dropout and Batch Normalization.

1.4.2 Stochastic Gradient Descent (SGD)

From a theoretical standpoint, our objective is to minimize the total loss over the entire dataset, which includes both the data loss and the regularization term:

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$

To update the weights, we need the exact gradient of this total loss:

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Computing this true gradient (the **Full Batch** gradient) requires evaluating the sum over all N samples in the dataset at every single step. When N is in the millions, this becomes computationally unfeasible.

Key Word

Stochastic Gradient Descent (SGD): Instead of the full sum, SGD approximates the true gradient by computing the sum over a small **minibatch** of examples (commonly 32, 64, or 128). This introduces noise but allows for much faster and frequent parameter updates.

1.5 Backpropagation and Neural Networks

1.5.1 Backpropagation

To update weights, we compute gradients via **Backpropagation**, which applies the chain rule from the output back to the inputs (backward pass).

Example

Patterns in gradient flow:

- **Add gate:** Acts as a gradient distributor.
- **Max gate:** Acts as a gradient router.
- **Mul gate:** Acts as a “swap multiplier”.
- **Copy gate:** Acts as a gradient adder.

Note

Backprop with Matrices: When inputs and weights are vectors or matrices, local gradients become *Jacobian matrices*. However, Jacobians are often sparse. Instead of explicitly forming them, we use implicit multiplication. A very useful trick to remember: the downstream gradient dL/dx **always has the same shape** as x !

1.5.2 Deeper Neural Networks

By stacking multiple layers and applying non-linear activation functions, we create multi-layer Neural Networks (e.g., $f = W_2 \max(0, W_1 x)$).

1.5.3 The Role of Activation Functions

An **activation function** is a mathematical “gate” applied to the output of a layer ($z = Wx + b$) before passing it to the next layer. Its primary role is to introduce **non-linearity** into the network.

Note

Why do we need Non-Linearity? If we only stacked linear layers without activation functions, the entire deep network would mathematically collapse into a single linear transformation. For example, a 2-layer linear network $f = W_2(W_1 x)$ can be rewritten as $f = W'x$ where $W' = W_2 W_1$. Non-linear activations allow the network to learn and approximate highly complex, non-linear boundaries.

Some of the most common activation functions include:

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$. It squashes numbers to the range $[0, 1]$. Historically popular, but suffers from the *vanishing gradient* problem.
- **tanh:** Squashes numbers to the range $[-1, 1]$. Like sigmoid, its gradients vanish for very large or very small inputs.
- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. It is the most widely used default choice because it is computationally efficient and greatly mitigates the vanishing gradient problem.
- **Leaky ReLU and ELU:** Variations of ReLU designed to allow a small, non-zero gradient when $x < 0$ to prevent “dead neurons”.

Bibliography

- [1] Fabio Galasso. *Recap on Image Classification and Neural Networks*. Advanced Machine Learning Course Slides, Sapienza Università di Roma. Sept. 2025.